COMPTE RENDU DE TP

Ingénierie des infrastructures Big Data et Cloud

TP3 — Déploiement cloud et CI/CD

Microsoft Azure | Docker | GitHub Actions

Application conteneurisée

Architecture managée, identité OIDC et monitoring

Établissement :	ENSET Mohammedia — UH2C
Filière :	II BDCC
Réalisé par :	Marouane Mounir
Encadrant :	Pr. Kamal EL GUEMMAT
Année universitaire :	2026–2027
Date :	22 octobre 2026

Adaptation Azure du TP initialement proposé sur Oracle Cloud Infrastructure (OCI).

Table des matières

1 Introduction et objectifs **2**
1.1 Environnement de travail . . . 2

2 Architecture technique Azure **3**

3 Préparation de l'abonnement et du projet **5**
3.1 Gestion budgétaire . . . 5
3.2 Justification des choix architecturaux . . . 5

4 Conteneurisation et validation locale **6**
4.1 Commandes de reproduction . . . 6

5 Registre privé Azure Container Registry **7**
5.1 Cycle de versionnement des images . . . 8

6 Déploiement managé avec Azure Container Apps **8**
6.1 Validation de l'identité managée applicative . . . 9

7 Automatisation CI/CD avec GitHub Actions **10**
7.1 Fédération OIDC et permissions . . . 10
7.2 Étendue des rôles et confiance OIDC . . . 10
7.3 Workflow as Code . . . 10
7.4 Validation de la livraison continue . . . 12
7.5 Traçabilité des déploiements . . . 12

8 Supervision, coûts et analyse des résultats **12**
8.1 Coûts effectivement remontés dans Azure . . . 14
8.2 Contrôle des métriques et des révisions . . . 15
8.3 Incidents rencontrés et résolution . . . 15
8.4 Analyse critique : limites et perspectives . . . 16

9 Preuves complémentaires et reproductibilité **17**
9.1 Contrôle des événements GitHub Actions . . . 17
9.2 Vérifications Azure CLI indépendantes . . . 17
9.3 Critères d'acceptation . . . 18

10 Bilan et conclusion **18**

Références **18**

1 Introduction et objectifs

Ce travail pratique porte sur le déploiement d'une application web conteneurisée et l'automatisation de sa livraison. Les consignes pédagogiques initiales prévoyaient une machine Ubuntu sur Oracle Cloud Infrastructure, puis un déploiement avec Docker et Jenkins. Pour explorer une approche plus proche des architectures *cloud native*, nous avons réalisé une adaptation sur Microsoft Azure, avec des services managés et un pipeline GitHub Actions.

Les objectifs sont : (i) valider localement une image Docker, (ii) la publier dans un registre privé, (iii) rendre l'application accessible en HTTPS, (iv) automatiser construction et déploiement après chaque commit sur `main`, et (v) superviser l'exécution tout en maîtrisant le coût dans le cadre d'un abonnement Azure for Students.

1.1 Environnement de travail

Abonnement	Azure for Students (crédit étudiant de 100 USD à l'activation)
Région	France Central
Groupe de ressources	`rg-tp3-marouane-mounir`
Dépôt Git	`https://github.com/Marouanemounir/tp4`
Registre	`acrtp3marouanemounir` (Basic)
Application	`ca-tp3-marouane-standard` (Container Apps)
Chaîne CI/CD	GitHub Actions, Azure CLI, OIDC

2 Architecture technique Azure

Le schéma suivant présente les services déployés et les services d’observabilité considérés dans le TP ; les fonctions de conservation persistante des logs n’ont pas toutes été activées. Il s’inspire du langage de diagrammes de référence Microsoft Azure Architecture Center, sans reproduire une image du site. Les flèches pleines montrent le chemin d’intégration/livraison et la flèche HTTPS le trafic utilisateur.

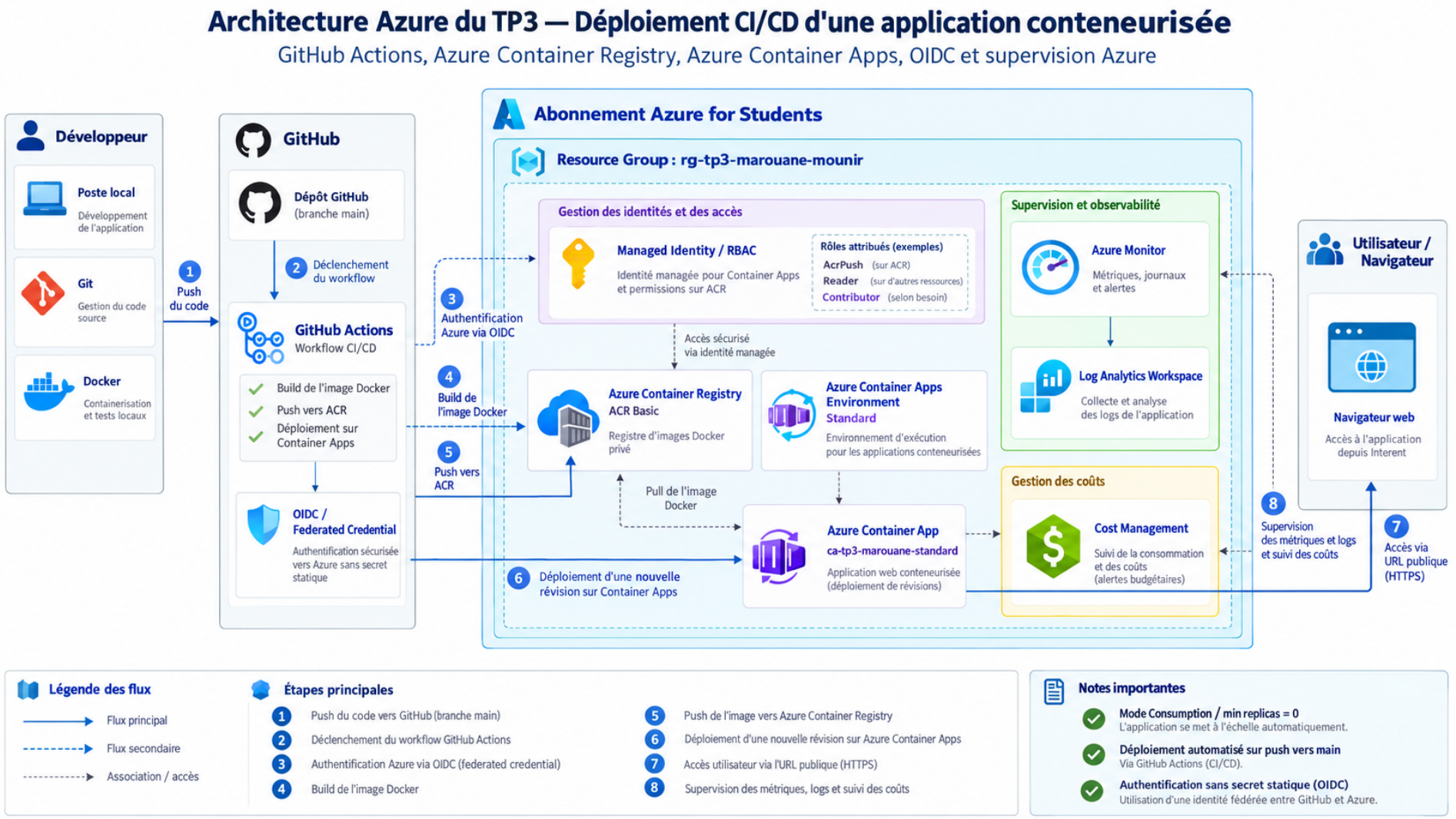


Figure 1 – Architecture Azure professionnelle du TP3 : flux CI/CD GitHub–ACR–Container Apps, authentification OIDC, contrôles RBAC, exposition HTTPS et observabilité. Les composants Log Analytics sont représentés dans le périmètre d'observabilité, leur collecte persistante ayant été limitée lors du TP.

Séparation des responsabilités : l'identité de l'application possède le droit `AcrPull` pour récupérer l'image. Une autre identité, dédiée à GitHub Actions, dispose de `AcrPush`, `Reader` sur ACR et `Contributor` limité à la Container App. La fédération OIDC évite l'emploi d'un secret client Azure permanent.

3 Préparation de l'abonnement et du projet

L'abonnement étudiant a été vérifié, ainsi que l'enregistrement des fournisseurs `Microsoft.Compute` et `Microsoft.App`. Les quotas de machines virtuelles classiques étaient à zéro dans France Central, ce qui a conforté le choix d'une solution managée. Le quota Container Apps a été confirmé à un environnement dans cette région.

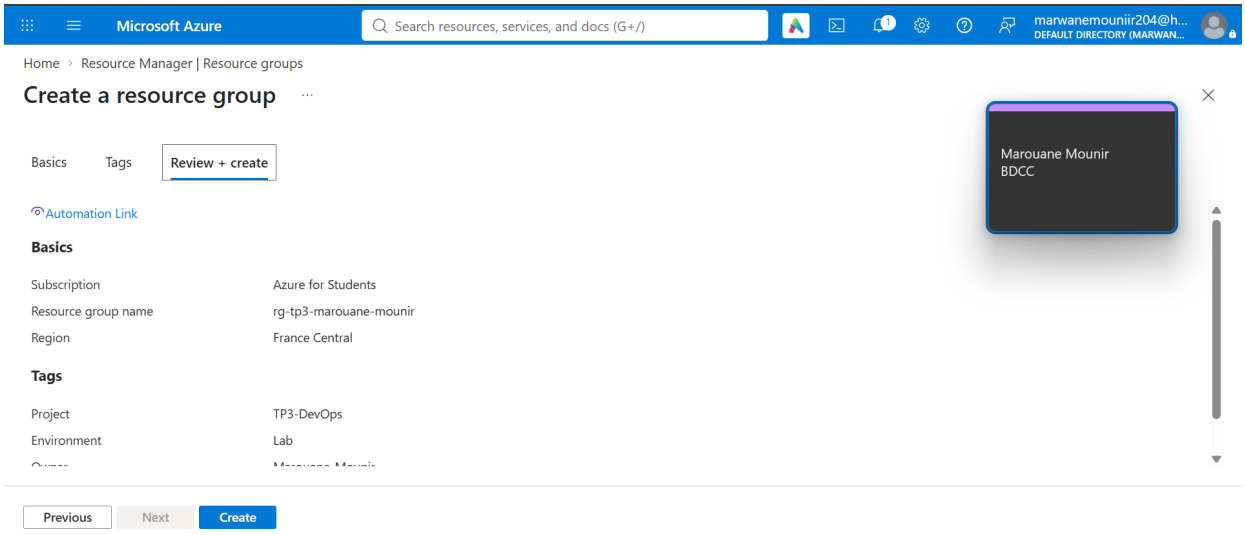


Figure 2 – Groupe de ressources du TP3 dans le portail Azure.

3.1 Gestion budgétaire

Un groupe de ressources spécifique isole les coûts du laboratoire. Les métriques de facturation peuvent être différées : une vue « No cost reported » ne prouve pas une absence de dépense. Le registre ACR Basic, notamment, demeure facturable durant son existence. Une politique d'arrêt/suppression en fin de TP est recommandée.

3.2 Justification des choix architecturaux

La chaîne de livraison sépare clairement les systèmes de développement, de stockage d'artefacts et d'exécution. Le registre privé ACR assure la conservation et la version des images ; Container Apps assure l'exécution de Nginx sans maintenance directe d'un serveur Linux. L'environnement Standard associé au profil Consumption fournit l'élasticité nécessaire à ce laboratoire. L'URL publique HTTPS expose seulement l'application, tandis que la publication d'images s'effectue par un canal authentifié.

La zone identité et accès distingue deux rôles : l'identité gérée par l'application lit l'image privée et l'identité fédérée de GitHub Actions publie les versions et actualise la configuration applicative. Les vues de surveillance et de coûts restent distinctes des flux de livraison.

4 Conteneurisation et validation locale

L'application de départ est la page web Nginx du TP2. Le dépôt contient `index.html` et un `Dockerfile`, avec une image Nginx et une copie du fichier HTML vers le répertoire web du conteneur. Le test local a été réalisé avec `docker build` et `docker run -p 8082:80`, puis dans le navigateur sur `localhost:8082`.

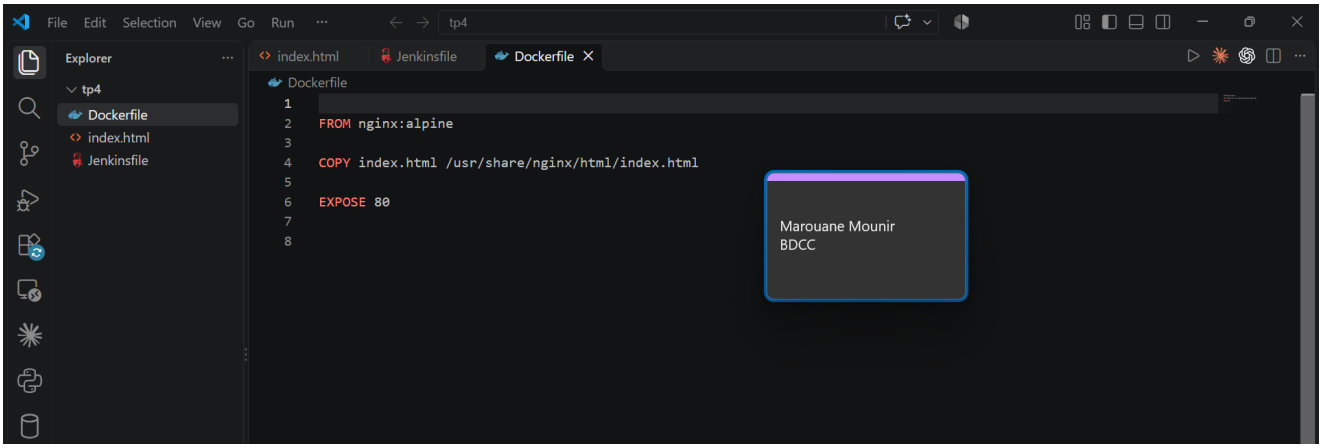


Figure 3 – Projet source et fichiers applicatifs utilisés pour la construction Docker.

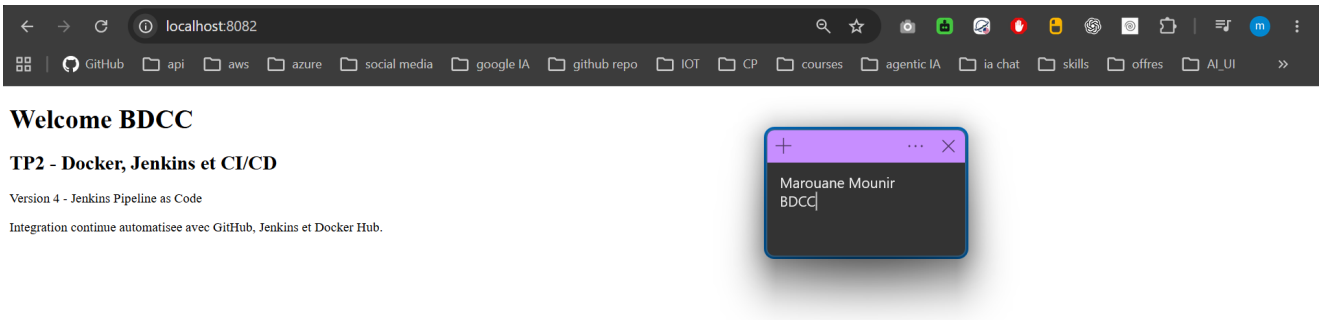


Figure 4 – Application Nginx validée localement avant publication dans le cloud.

4.1 Commandes de reproduction

```
docker build -t tp3-azure:v1 .
docker run -d --name tp3-azure -p 8082:80 tp3-azure:v1
docker ps
```

Le test local permet de distinguer les problèmes applicatifs des problèmes d'authentification, de réseau et de permissions liés au cloud. La page Nginx s'affichant sur le port local 8082, l'image pouvait être promue vers un registre privé.

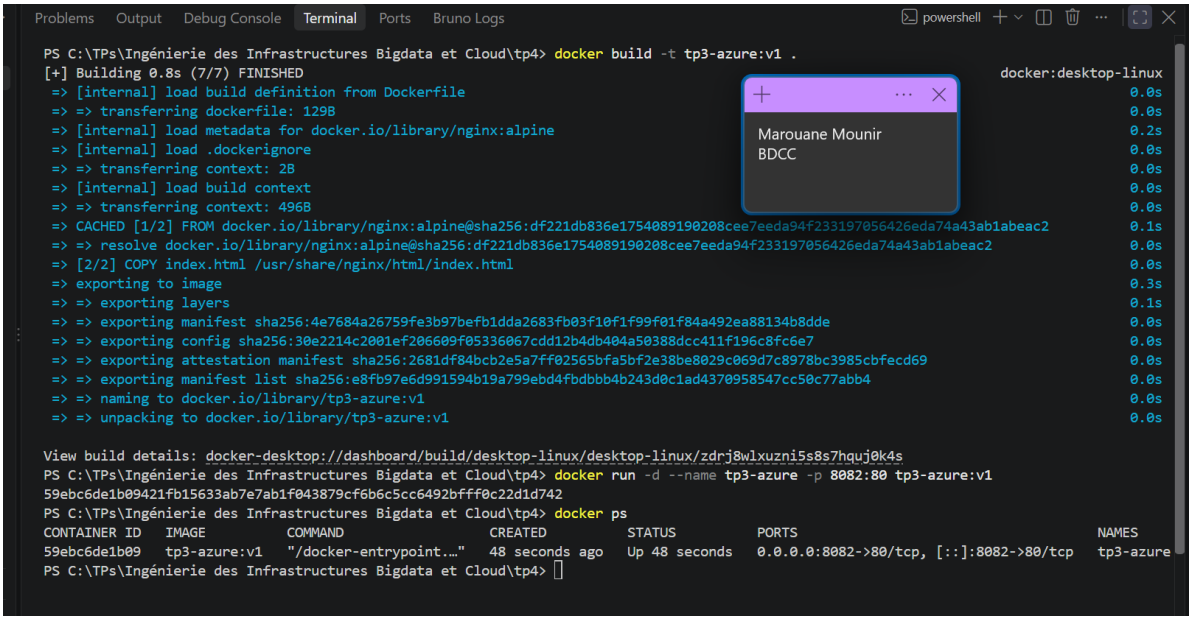


Figure 5 – Construction locale réussie de l'image Docker et vérification du conteneur en exécution.

5 Registre privé Azure Container Registry

Un registre Azure Container Registry **Basic** a été créé dans France Central. L'image locale a été taguée puis publiée dans le dépôt `tp3-azure` avec le tag initial `v1`. Les versions issues du pipeline sont ensuite identifiées par le SHA Git, ce qui facilite la traçabilité et évite de dépendre d'un tag mouvant `latest`.

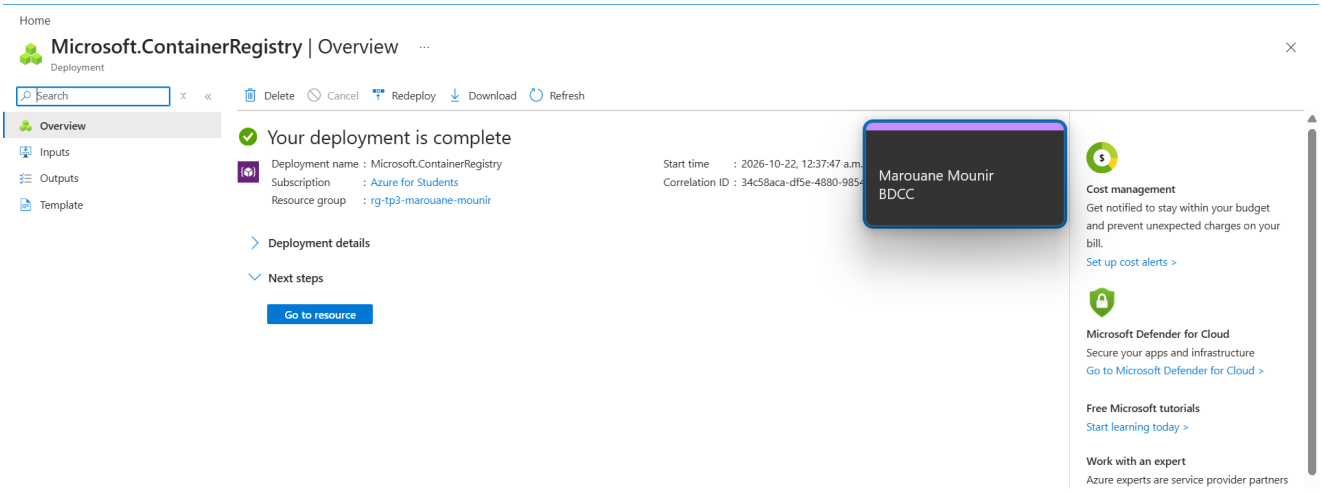


Figure 6 – Vue du registre ACR privé employé pour les images de l'application.

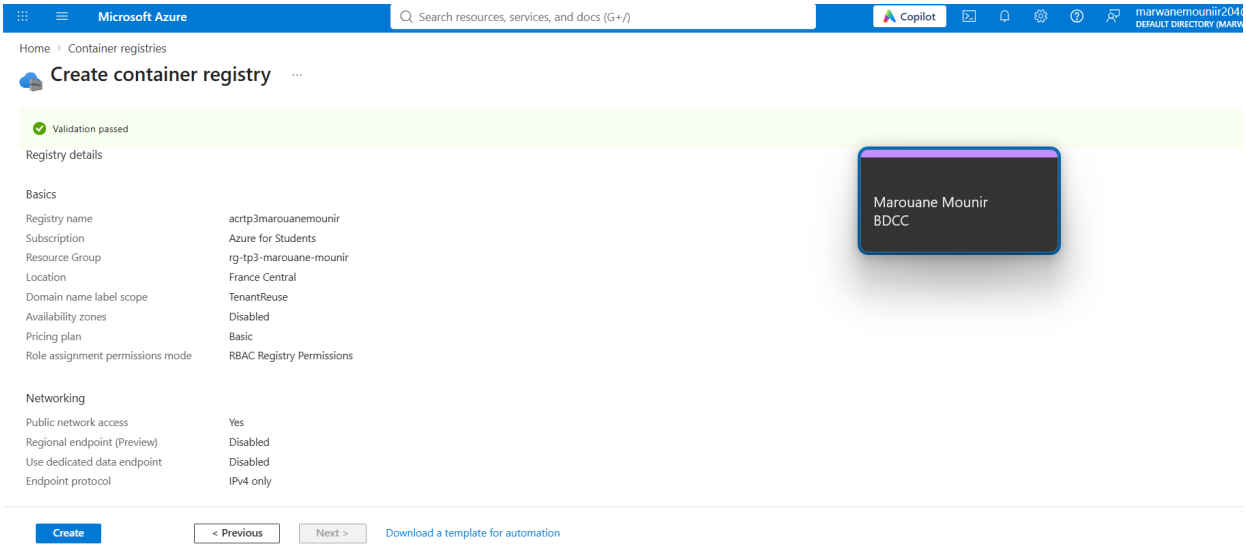


Figure 7 – Création d'Azure Container Registry dans le groupe de ressources du TP3 avec le niveau Basic.

5.1 Cycle de versionnement des images

La première version a été publiée sous `tp3-azure:v1` depuis le poste de développement. Le pipeline CI/CD utilise ensuite le SHA intégral du commit comme tag. Ainsi, chaque version déployée est rattachée de manière reproductible à un état du code source. La permission `AcrPush` autorise l'identité du pipeline à publier les images, tandis que `AcrPull` est limité à leur récupération lors de l'exécution.

6 Déploiement managé avec Azure Container Apps

Le premier essai s'est effectué dans un environnement *Express*. Cette configuration a empêché l'authentification ACR par identité système, conduisant à un échec de récupération de l'image privée. Pour conserver une authentification sans mot de passe, l'environnement Express a été supprimé et remplacé par un environnement **Standard**, profil **Consumption**, dans France Central.

Les paramètres retenus incluent l'accès externe HTTPS, le port cible 80, une réplication minimale de 0 et maximale de 1, ainsi qu'une image privée ACR. Une identité système affectée à l'application a reçu le rôle `AcrPull`. L'application a ensuite été vérifiée sur son adresse publique.

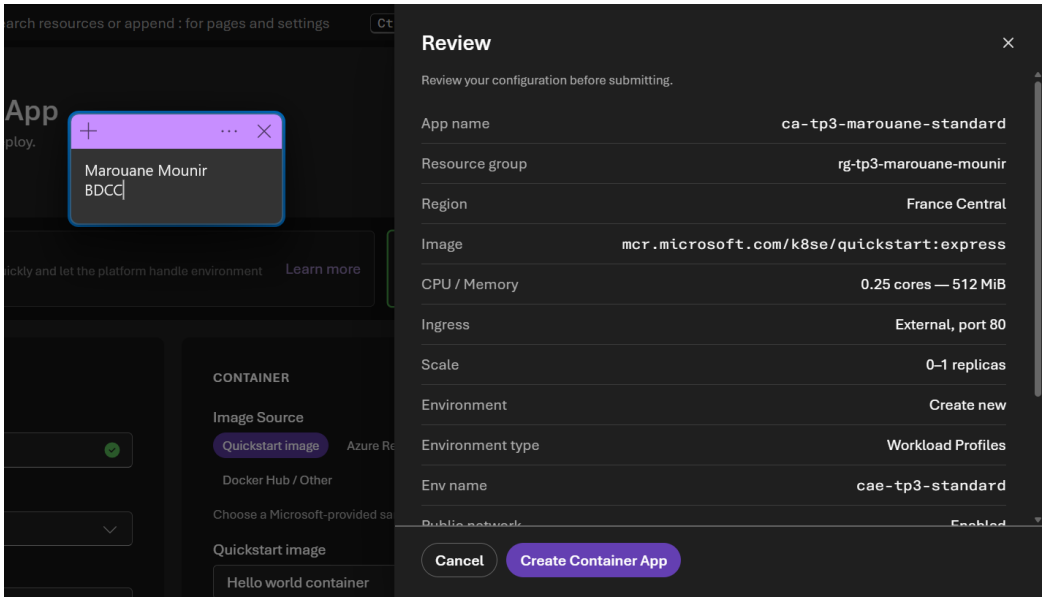


Figure 8 – Création de l'application Standard et paramètres d'environnement.

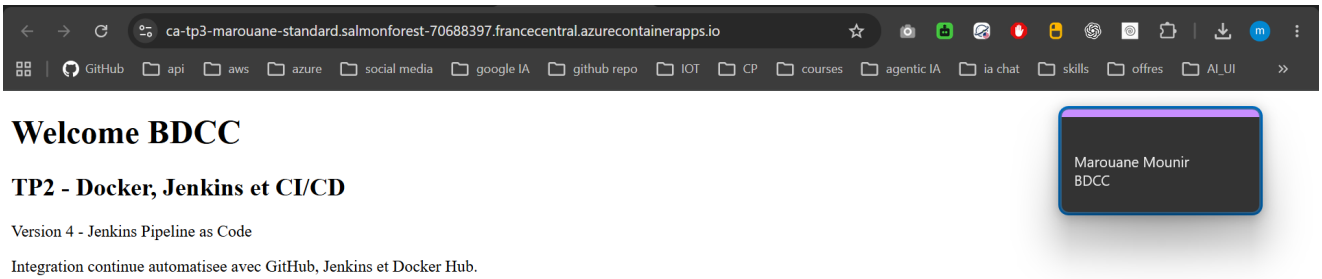


Figure 9 – Application du TP2 rendue disponible via une URL HTTPS Azure.

6.1 Validation de l'identité managée applicative

L'identité `SystemAssigned` est générée pour la nouvelle Container App Standard. Elle est associée au contrôle d'accès du registre ACR : l'application peut récupérer la version publiée sans qu'un mot de passe de registre soit injecté dans le conteneur. Cette approche limite l'exposition des secrets et facilite leur rotation implicite par la plateforme.

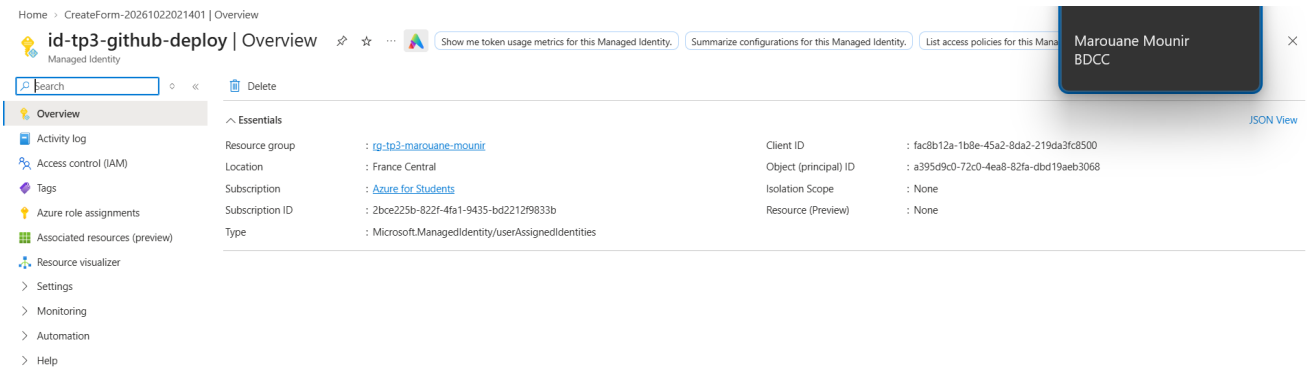


Figure 10 – Activation de l'identité managée de l'application pour l'accès sécurisé à ACR.

7 Automatisation CI/CD avec GitHub Actions

7.1 Fédération OIDC et permissions

Une identité Azure dédiée à GitHub Actions a été créée. Le dépôt `Marouanemounir/tp4` et sa branche `main` ont été déclarés dans un *federated credential* émis par `token.actions.githubusercontent.com`, avec pour audience `api://AzureADTokenExchange`. Les trois identifiants de configuration (client, tenant, abonnement) sont définis dans les secrets GitHub Actions, sans mot de passe Azure permanent.

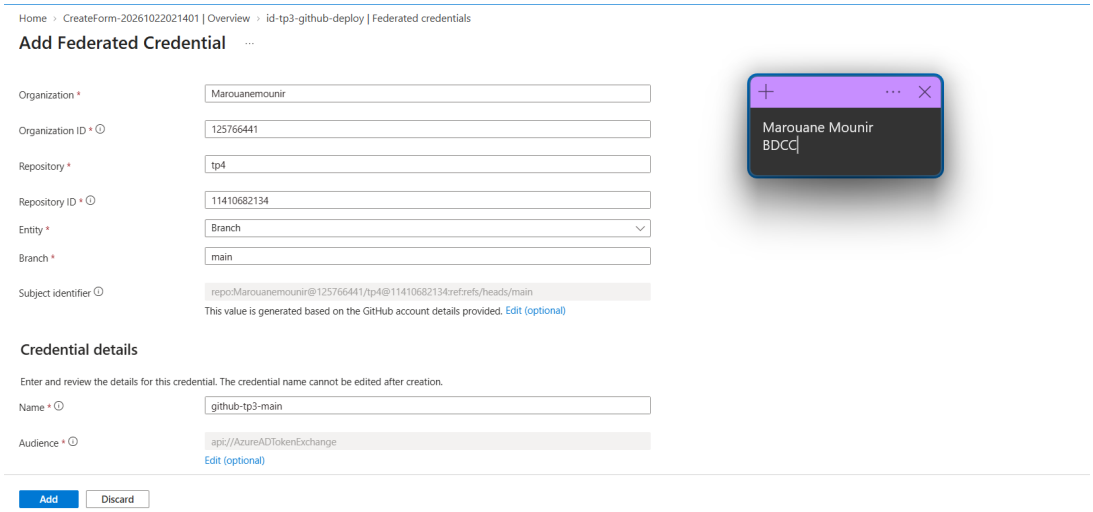


Figure 11 – Paramétrage de l'identité fédérée OIDC pour le dépôt GitHub.



Figure 12 – Vérification des rôles Azure nécessaires au pipeline.

7.2 Étendue des rôles et confiance OIDC

L'authentification GitHub repose sur une fédération de jetons temporaires : le jeton émis pour la branche `main` doit correspondre exactement au sujet déclaré dans Azure, y compris les identifiants immuables de l'organisation et du dépôt. Les rôles ont été associés à des périmètres précis : `AcrPush` et `Reader` au registre ACR ; `Contributor` à la Container App cible. Les données de configuration utilisées par `azure/login` figurent dans les secrets GitHub, sans mot de passe client permanent.

7.3 Workflow as Code

Le fichier `.github/workflows/azure-deploy.yml` est exécuté à chaque `push` sur la branche `main`. Il valide les fichiers, s'authentifie avec OIDC, récupère le nom DNS du registre, construit une image Docker, la pousse dans ACR et met à jour la Container App. Une vérification finale compare l'image configurée à celle correspondant au commit.

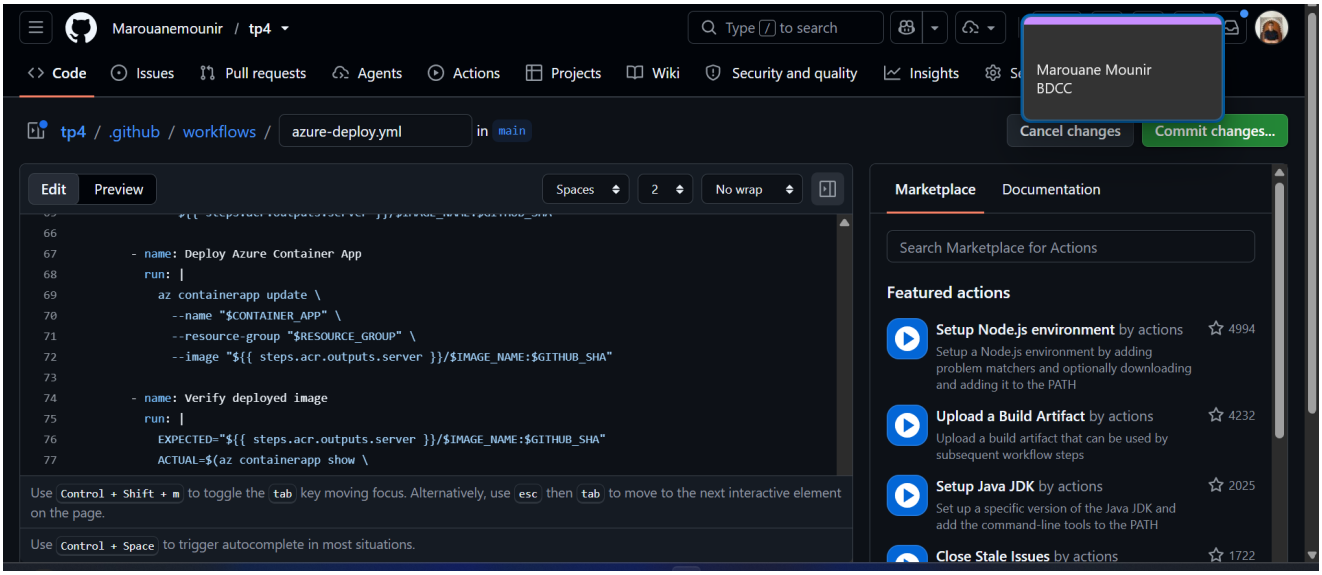


Figure 13 – Pipeline CI/CD défini en YAML et versionné dans GitHub.

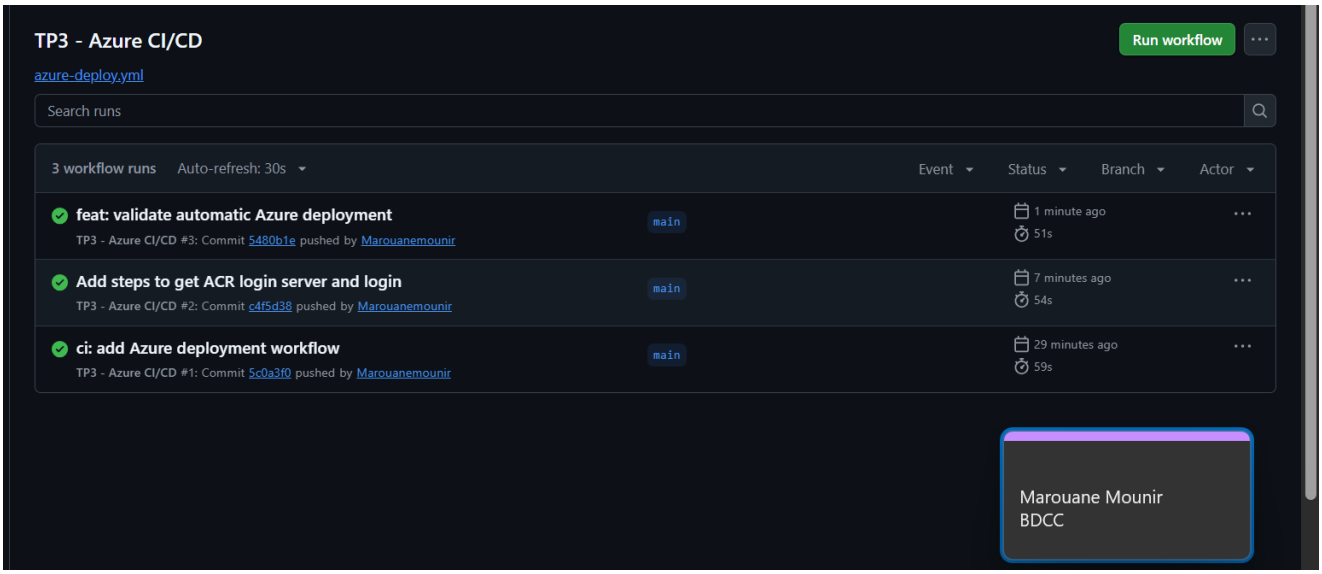


Figure 14 – Exécution GitHub Actions entièrement réussie : build, push et déploiement.

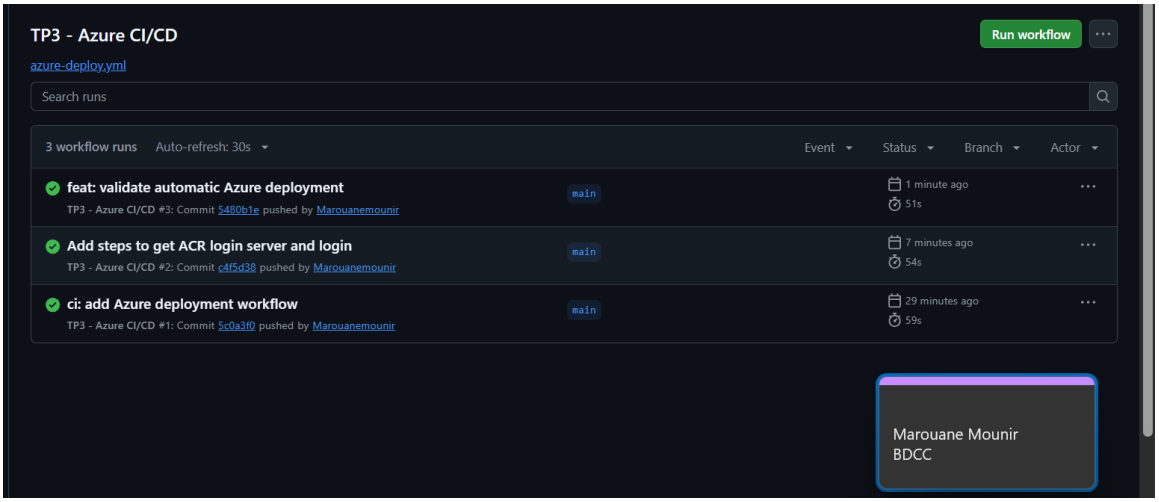


Figure 15 – Historique GitHub Actions : distinction entre premiers essais de configuration et exécution validée.

7.4 Validation de la livraison continue

Une modification du fichier `index.html` a généré un nouveau commit sur `main`, entraînant automatiquement un nouveau build et une nouvelle révision Azure Container Apps. La version 5, mentionnant Azure CI/CD avec GitHub Actions, a été observée sur l'URL HTTPS publique. Le test prouve le fonctionnement de la chaîne CI/CD, au-delà d'une simple exécution manuelle.

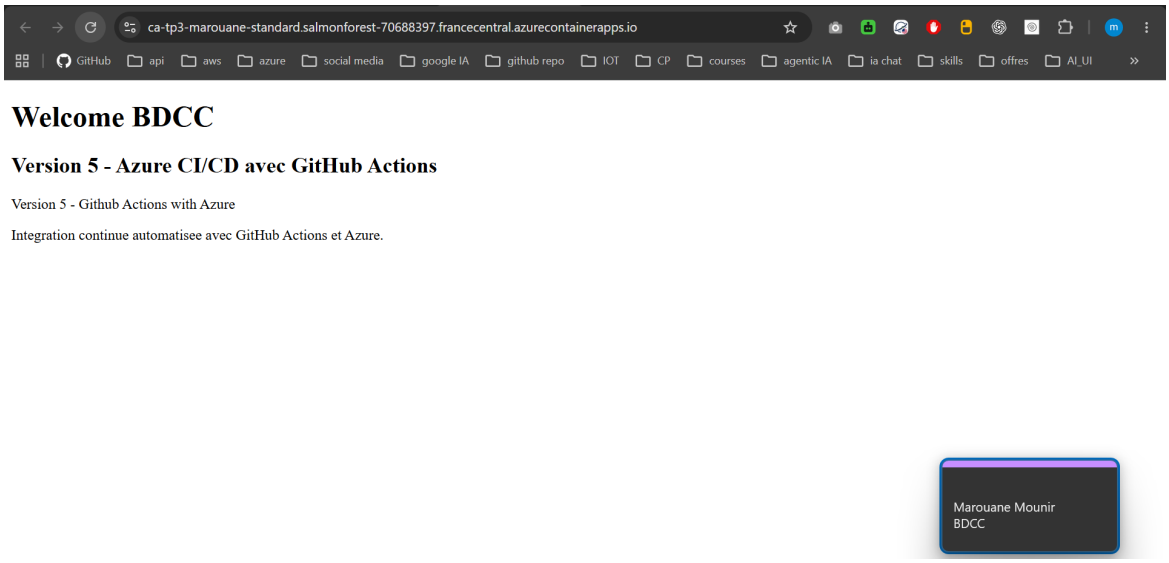


Figure 16 – Version 5 de la page Nginx publiée automatiquement après un commit GitHub.

7.5 Traçabilité des déploiements

La validation finale ne se limite pas à la couleur verte du workflow. Après le commit de validation, la nouvelle version est visible dans la page publique et la Container App référence l'image issue du commit. Ce recoupement entre GitHub, ACR, révision et contenu web établit la continuité de livraison de bout en bout.

8 Supervision, coûts et analyse des résultats

Le suivi opérationnel repose sur la gestion des révisions, les métriques de consommation et le flux de journaux. Le paramètre `minReplicas=0` autorise l'arrêt automatique des réplicas inactifs ; un état `ScaledToZero` ne représente pas nécessairement un incident. Les logs n'ont pas été configurés pour un stockage persistant au départ afin de limiter les coûts ; le streaming reste utilisé pour le diagnostic ponctuel.

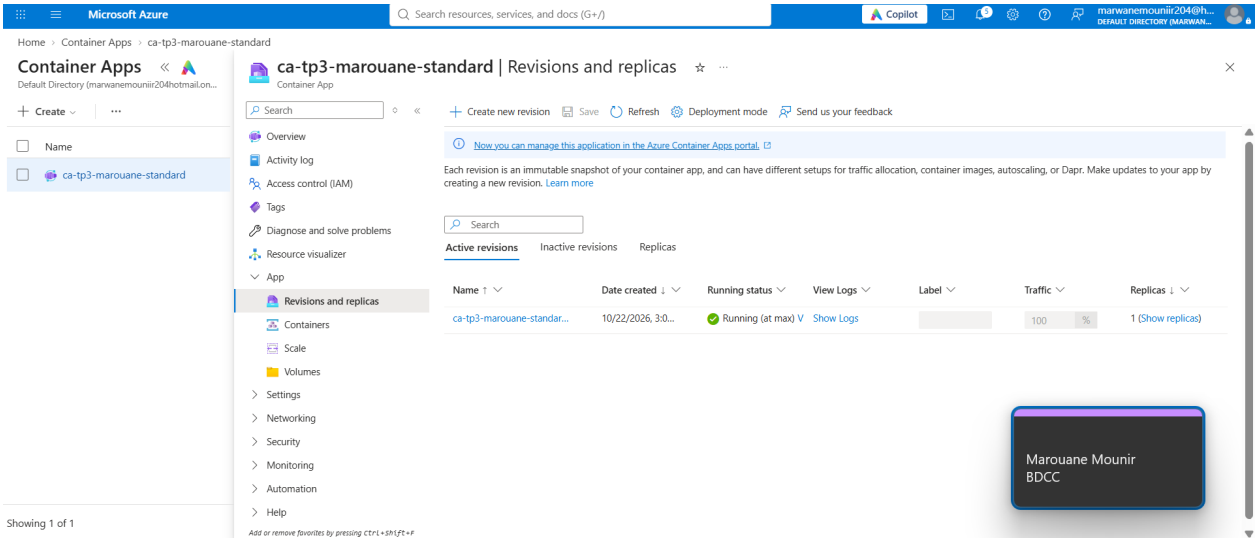


Figure 17 – Contrôle des révisions de la Container App déployée.

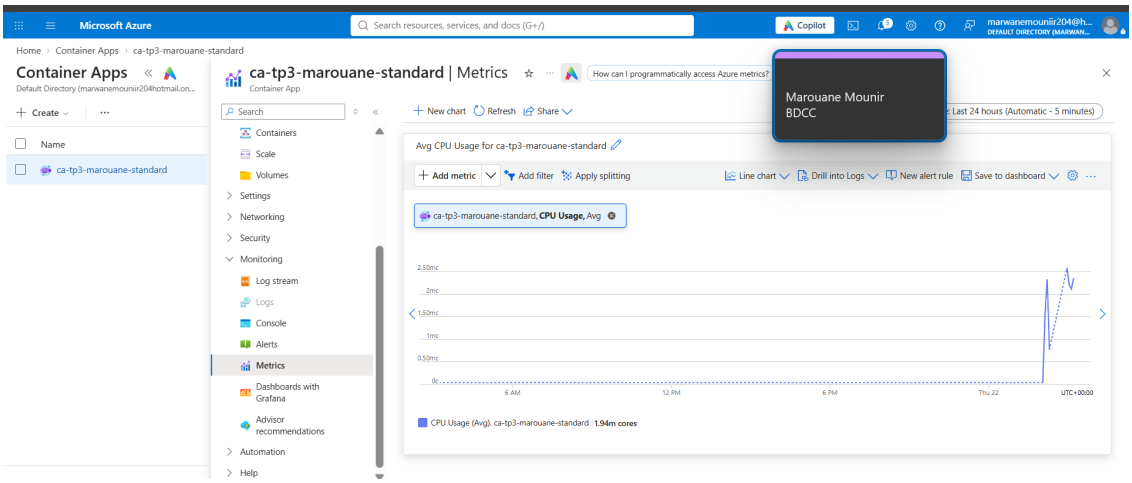


Figure 18 – Observation des métriques de l'application Azure dans le portail.

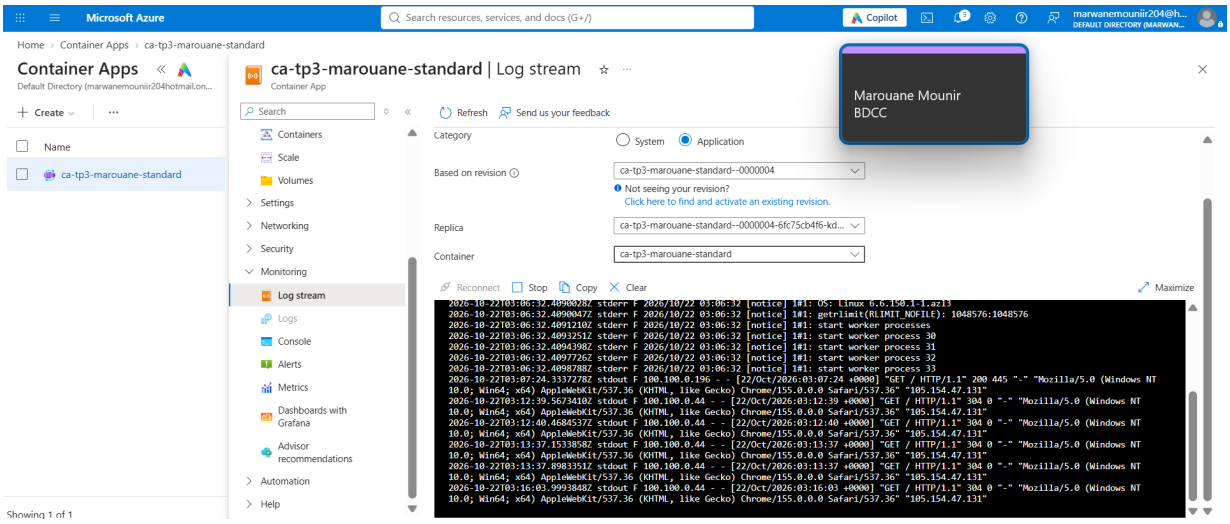


Figure 19 – Consultation des journaux de l'application et diagnostic en temps réel.

8.1 Coûts effectivement remontés dans Azure

Le relevé de facturation daté du 22 octobre 2026 affiche un coût cumulé de **0,05 USD** au niveau du compte de facturation, et un budget **mensuel de 10 USD**. Les montants visibles par service sont : **Container Registry : 0,04 USD**, **Container Apps : moins de 0,01 USD** et **Log Analytics : 0,00 USD**. Ces chiffres sont une photographie à la date de capture et non une estimation du coût final. La ventilation affichée est au niveau du compte de facturation ; elle ne doit pas être confondue avec une mesure exclusivement imputable au groupe de ressources.

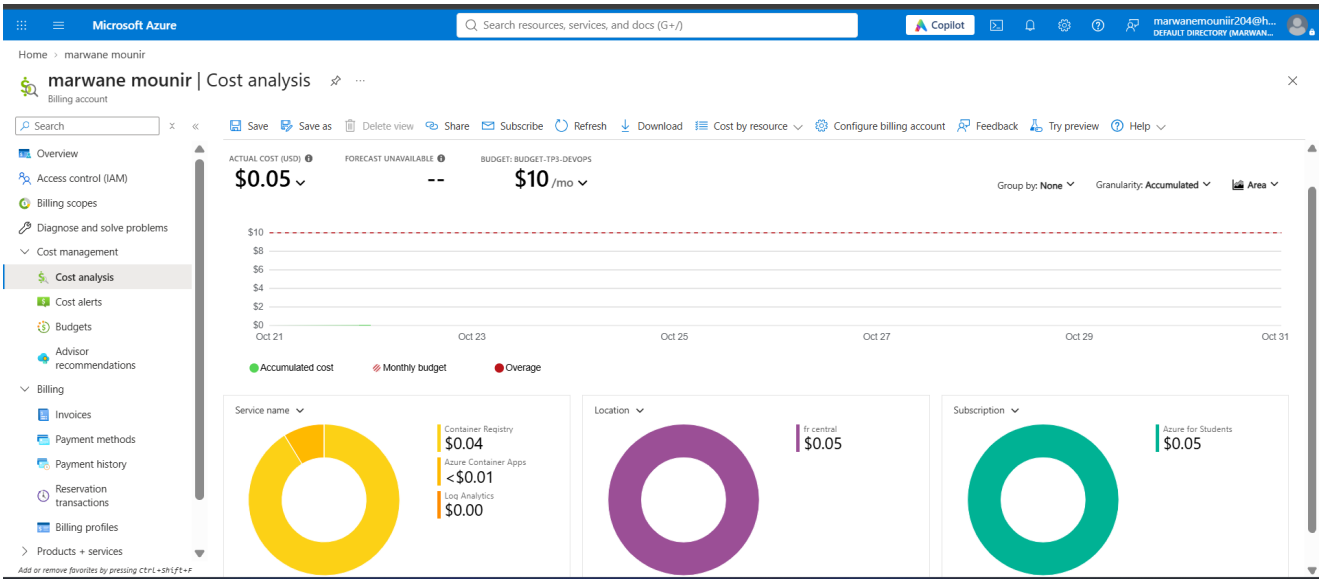


Figure 20 – Coût effectivement remonté le 22 octobre 2026 : 0,05 USD ; budget de 10 USD/mois ; ventilation par service.

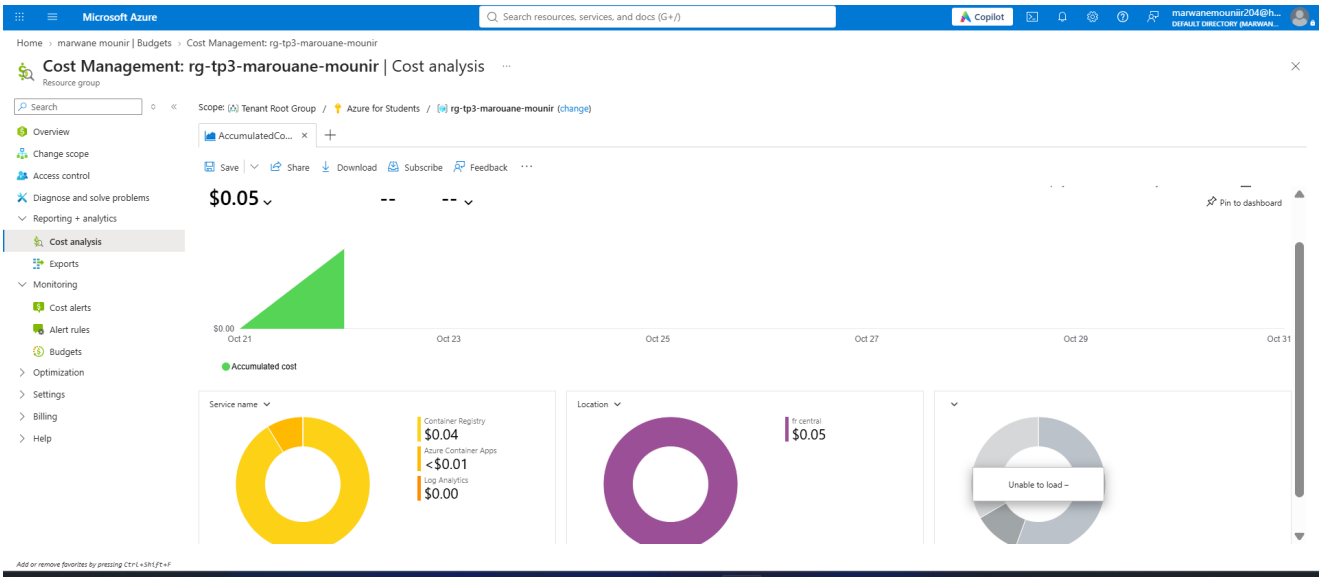


Figure 21 – Autre vue du suivi de consommation et de la répartition des dépenses Azure.

Lecture critique. Une autre capture du périmètre `rg-tp3-marouane-mounir` indiquait auparavant « No cost reported ». Cette absence de données tenait au périmètre et au délai de remontée de la facturation ; elle n'est pas incompatible avec la capture ultérieure à 0,05 USD au niveau du compte.

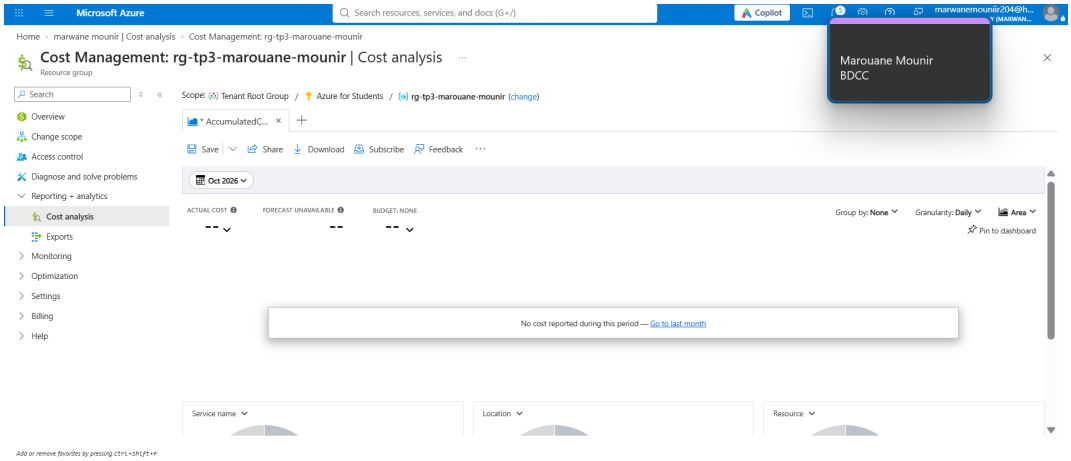


Figure 22 – Vue initiale Cost Analysis du groupe de ressources, avant remontée des données de facturation.

8.2 Contrôle des métriques et des révisions

Les graphiques du portail ont permis d'observer les requêtes et l'utilisation des ressources après consultation de l'application publique. L'existence d'une révision `Provisioned` ou d'un état `ScaledToZero` doit être interprétée conjointement avec la santé applicative et un accès HTTPS réel. La configuration initiale sans conservation systématique des journaux illustre également le compromis coût–observabilité d'un TP étudiant.

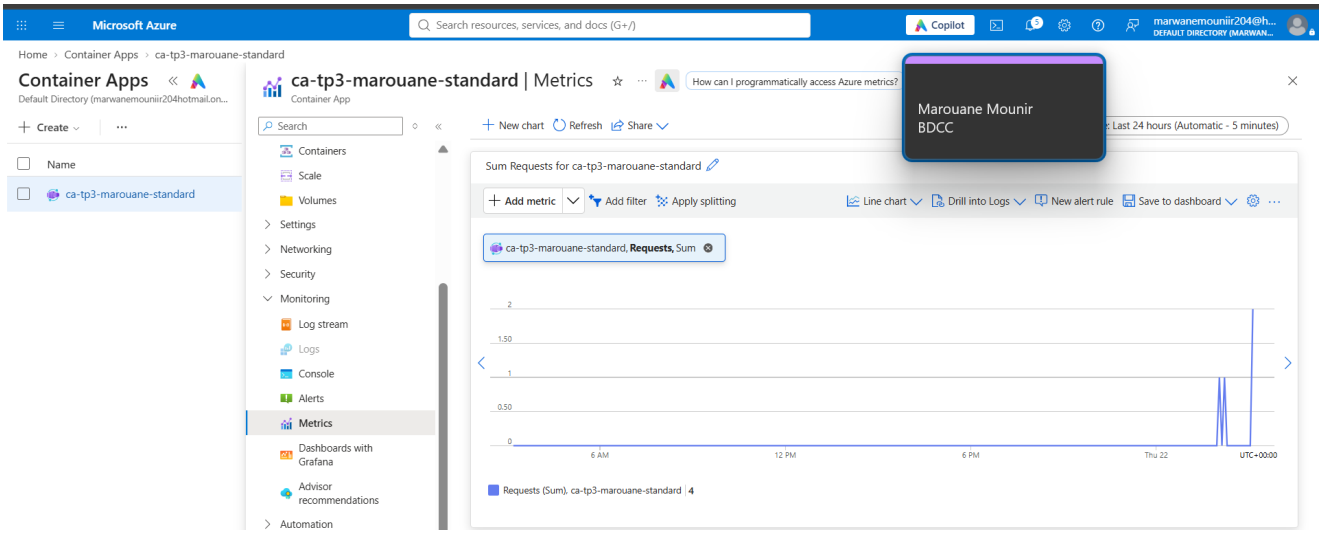


Figure 23 – Vue complémentaire des métriques de Container Apps pendant la validation.

8.3 Incidents rencontrés et résolution

Incident	Action corrective et enseignement
Quota Compute nul	Orientation vers Container Apps plutôt qu'une VM dédiée.
Environnement Express et ACR	Migration vers l'environnement Standard pour une authentification par identité système.
Erreur OIDC AADSTS700213	Alignement exact du sujet fédéré avec le sujet du jeton GitHub (identifiants numériques).
ACR introuvable dans le workflow	Vérification des rôles Reader/AcrPush et précision du groupe de ressources dans la commande CLI ; l'exécution a ensuite réussi.

8.4 Analyse critique : limites et perspectives

Le périmètre couvre la livraison continue d'une page statique Nginx mais pas les exigences complètes d'une chaîne de production. Les contrôles de santé HTTP, analyses de vulnérabilités d'images, tests applicatifs systématiques, stratégies de retour arrière et validations multi-environnements sont des pistes d'amélioration. L'usage de composants managés économise l'administration d'une VM, mais nécessite une maîtrise explicite des rôles Azure et de la facturation des services persistants.

9 Preuves complémentaires et reproductibilité

9.1 Contrôle des événements GitHub Actions

L'historique du dépôt permet de retracer la progression depuis les premiers essais jusqu'à l'exécution complète de la chaîne. La lecture de l'onglet Actions permet de vérifier l'ordre des opérations, le résultat de chaque job et la durée totale de l'exécution. Les erreurs d'authentification observées lors du paramétrage OIDC ont été corrigées sans stocker de secret Azure permanent.

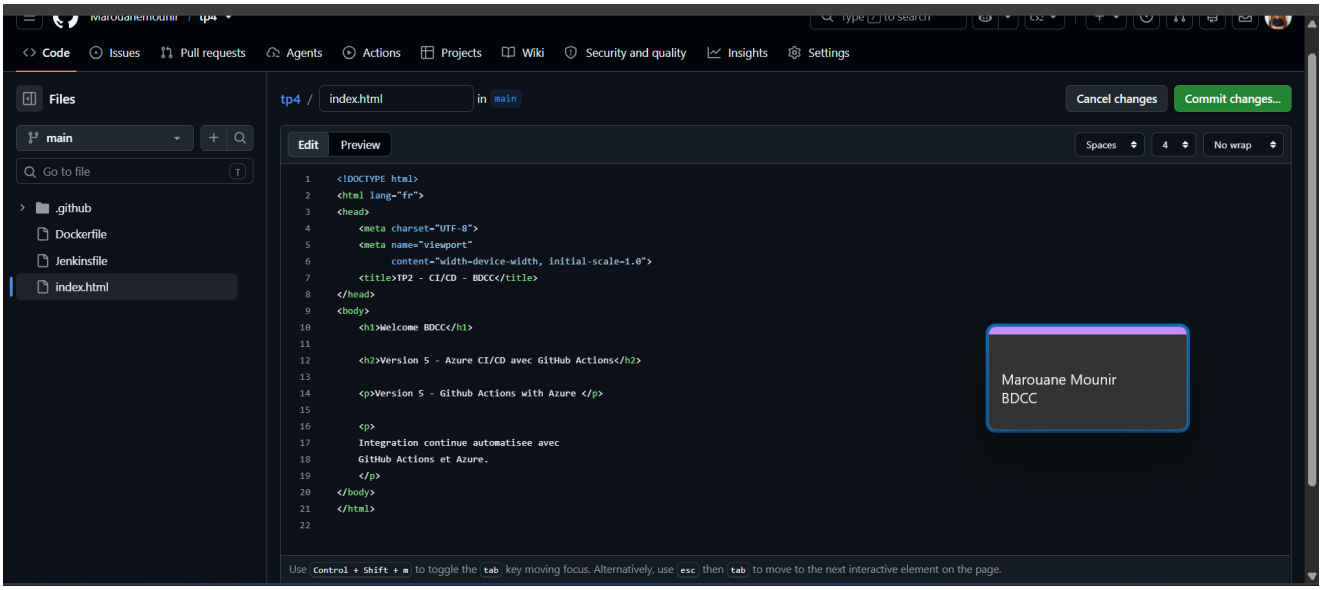


Figure 24 – Configuration du workflow GitHub Actions utilisée pour la livraison continue.

9.2 Vérifications Azure CLI indépendantes

La commande `az containerapp revision list` confirme la présence d'une révision gérée par la plateforme. La comparaison du nom de l'image configurée avec le SHA du commit vérifie la cohérence entre le dépôt source, le registre et la ressource d'exécution. Ces contrôles, répétés après chaque changement significatif, permettent de détecter un déploiement partiellement réussi malgré la présence d'un message de succès au niveau du pipeline.

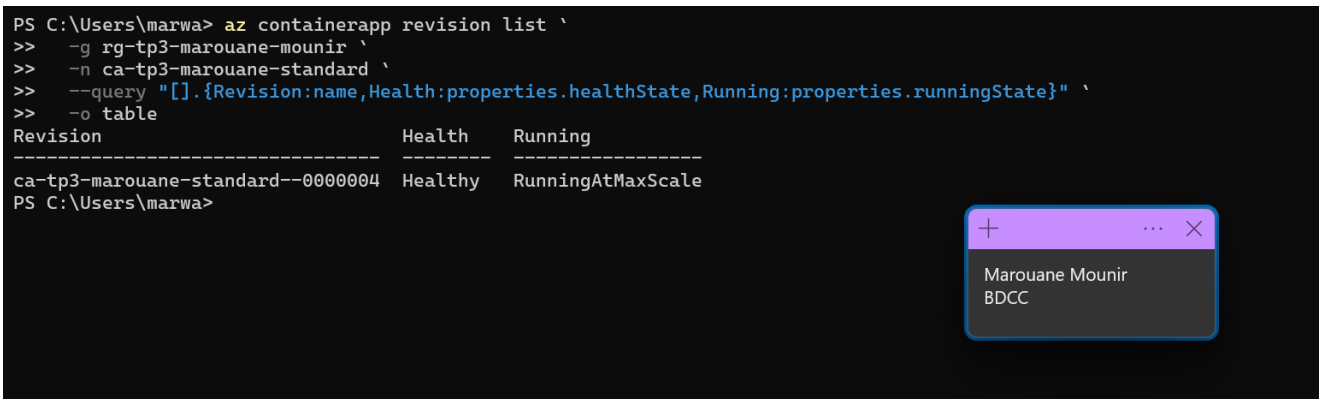


Figure 25 – Vérification des révisions et des états depuis Azure CLI.

9.3 Critères d'acceptation

— La page personnalisée s'affiche en HTTPS dans un navigateur sans intervention manuelle sur un serveur.

— La construction et la publication Docker s'effectuent après un push sur la branche principale.

— Le nouveau tag d'image renvoie au SHA Git de la version livrée.

— Le registre ne nécessite pas d'identifiants administrateur dans le workflow ; le contrôle d'accès utilise OIDC et RBAC.

— Le nombre de réplicas peut revenir à zéro sans entraîner la suppression de l'application.

10 Bilan et conclusion

Le TP3 a permis de réaliser un cycle complet de déploiement et de livraison continue d'une application web conteneurisée. Le code source est hébergé dans GitHub ; le pipeline GitHub Actions produit une image Docker versionnée, la publie dans Azure Container Registry et déploie une nouvelle révision dans Azure Container Apps. La chaîne OIDC et les identités managées limitent le recours à des mots de passe permanents.

Par rapport aux consignes OCI/Jenkins, cette réalisation privilégie une **architecture Azure managée**. Ce choix supprime l'administration d'une VM et apporte une expérience pratique de CI/CD cloud native, avec surveillance et maîtrise budgétaire. Les améliorations envisageables sont l'ajout de tests unitaires, d'un scan de sécurité des images, d'un contrôle de santé HTTP et d'une stratégie de validation avant production.

Références

1. Pr. Kamal EL GUEMMAT, *TP3 : Déploiement d'une application web sur OCI avec Jenkins*, support ENSET, 2026–2027.
2. Pr. Kamal EL GUEMMAT, *Guide TP3 : OCI DevOps et CI/CD*, support ENSET, 2026–2027.
3. Microsoft Learn, *Azure Architecture Center*, https://learn.microsoft.com/fr-fr/azure/architecture/browse/.
4. Microsoft Learn, *Déployer avec GitHub Actions dans Azure Container Apps*, https://learn.microsoft.com/fr-fr/azure/container-apps/github-actions.
5. Microsoft Learn, *Identités managées dans Azure Container Apps*, https://learn.microsoft.com/fr-fr/azure/container-apps/managed-identity.
6. Microsoft Learn, *Azure Well-Architected — Container Apps*, https://learn.microsoft.com/en-us/azure/well-architected/service-guides/azure-container-apps.